

Testing Policy

PenFlow by The BroCode

Update date: 2026/09/04

Contents

1. Introduction
2. Unit and Integration Testing Policies
 - 2.1 Unit Testing Policy
 - 2.1.1 Objective
 - 2.1.2 Automation
 - 2.1.3 Coverage
 - 2.1.4 Framework
 - 2.2 Integration Testing Policy
 - 2.2.1 Objective
 - 2.2.2 Automation
 - 2.2.3 Framework
 - 2.3 Test Naming Conventions
3. Testing Workflow
4. Quality Assurance Testing
 - 4.1 Performance
 - 4.1.1 Objective
 - 4.1.2 Performance Metrics
 - 4.1.3 Testing Framework
 - 4.1.4 Test Scenarios
 - 4.1.5 Acceptance Criteria
 - 4.2 Scalability
 - 4.2.1 Objective
 - 4.2.2 Scalability Metrics
 - 4.2.3 Testing Framework
 - 4.2.4 Load Testing Scenarios
 - 4.2.5 Performance Thresholds
 - 4.3 Security
 - 4.3.1 Objective
 - 4.3.2 Security Test Coverage
 - 4.3.3 Testing Framework
 - 4.3.4 Security Test Scenarios
 - 4.3.5 Security Acceptance Criteria

5. End-to-End Testing Policy

- 5.1 Objective
- 5.2 Framework
- 5.3 Test Scenarios
- 5.4 Acceptance Criteria

6. Defect Management

- 6.1 Defect Reporting
- 6.2 Severity Classification

1. Introduction

This document outlines the Testing Policy for PenFlow, a cybersecurity SaaS platform for penetration testing workflow management, developed by The BroCode for BlueVision (Practical Cyber Security). The purpose of this policy is to establish a clear framework for testing procedures, ensuring that all aspects of the application are thoroughly evaluated before demonstration and deployment.

The testing policy includes, but is not limited to:

- Unit testing
- Integration testing
- End-to-end testing
- Performance testing
- Scalability testing
- Security testing

The sections below provide detailed procedures, tools, and responsibilities associated with each testing phase, ensuring that all components of PenFlow are validated before each demo submission. For Demo 2, at least five fully implemented use cases must be supported by passing unit, integration, and end-to-end tests with no mocks in the final build.

2. Unit and Integration Testing Policies

This section details the testing policy for the PenFlow development project, focusing on unit testing and integration testing. Automated testing is used to enhance efficiency and accuracy. Both unit and integration tests are integrated into the CI pipeline, which is configured using GitHub Actions. Automated testing is a key component in the pipeline, preventing code from being merged into the main branch if tests fail. This approach maintains high standards of code quality. To maintain a robust testing framework, a minimum test coverage of 80% is targeted across all testable layers of the project. This policy must be implemented by all contributors and is designed to facilitate continuous improvement in the application.

2.1 Unit Testing Policy

2.1.1 Objective

The objective of unit testing is to validate the functionality of individual components within the software, ensuring that each section operates as intended. This includes testing function logic, validation rules, error handling, and data transformation in isolation from external dependencies such as the database, message broker, and third-party APIs.

2.1.2 Automation

All unit tests are automated and integrated into the GitHub Actions pipeline. This enforces testing protocols and prevents any code from being merged into other branches if tests fail. Unit tests run on every push to a feature branch and on every pull request targeting the main or develop branch.

2.1.3 Coverage

A minimum code coverage of 80% is targeted for unit testing across all backend services, Celery workers, and frontend components. All service layer functions and utility helpers must undergo unit testing. Coverage reports are generated on every CI run using `pytest-cov` for Python modules and Jest with the `--coverage` flag for TypeScript modules, and are uploaded as CI artifacts.

2.1.4 Framework

The following frameworks are used for unit testing across the system:

- **Backend (Python):** `pytest` with `pytest-mock` and `unittest.mock` for mocking external dependencies including database sessions, HTTP clients, and third-party API responses.
- **Frontend (TypeScript):** Jest with `@testing-library/react` for testing React component behaviour, form validation logic, and UI state transitions.

2.2 Integration Testing Policy

2.2.1 Objective

The objective of integration testing is to validate interactions between multiple system layers, ensuring that they function together as expected. Integration tests cover the full request lifecycle from HTTP request through schema validation, service logic, repository, and database response. Only external third-party APIs (Shodan, HIBP, crt.sh) are mocked; all internal layers interact with a live test database instance.

2.2.2 Automation

All integration tests are automated and integrated into the GitHub Actions pipeline. They run after unit tests pass, using a PostgreSQL test container provisioned by the CI environment. Tests that fail prevent the pull request from being merged.

2.2.3 Framework

The following frameworks are used for integration testing:

- **Backend (Python):** `pytest` with `httpx.AsyncClient` to issue HTTP requests against the FastAPI application, connected to a dedicated `penflow_test` PostgreSQL database that is seeded before each suite run and torn down after.

- **Workers (Python):** pytest with Celery configured in ALWAYS_EAGER mode to execute worker tasks synchronously during tests without requiring a running RabbitMQ broker.

2.3 Test Naming Conventions

Test files and test functions follow consistent naming patterns across the codebase to ensure maintainability and clear traceability between tests and the functionality they validate.

File Naming

- **Backend (Python/pytest):** Test files are named using the pattern `test_<module_or_feature>.py`, where `<module_or_feature>` corresponds to the component, service, or repository being tested. Test files are organised under the `tests/` directory structure with separate subdirectories for unit tests and integration tests.
- **Frontend (TypeScript/Jest):** Test files are named using the pattern `<name>.test.ts`, where `<name>` corresponds to the component, utility, or module being tested. Component tests use the `.tsx` extension.
- **E2E (Cypress):** End-to-end test files are named using the pattern `<use_case>_e2e.cy.ts`, where `<use_case>` describes the user journey being tested. Use case identifiers from the test scenarios table may be used as prefixes for traceability.

Test Function and Case Naming

- **Python (pytest):** Test functions are named in `snake_case` following the pattern `test_<action>_<scenario_or_outcome>`. This format describes the behaviour being tested and its expected result, where `<action>` indicates the operation performed and `<scenario_or_outcome>` describes the specific test condition or expected result.
- **Frontend (Jest) and E2E (Cypress):** Tests use `describe()` blocks grouped by feature or component, with `it()` statements written as plain-English descriptions of expected behaviour in the present tense. Each `it()` statement must clearly communicate the specific behaviour being verified without requiring additional documentation.

3. Testing Workflow

The testing workflow outlines the systematic approach taken to ensure the quality of the PenFlow software. The steps in the workflow are as follows:

- **Development:** When a developer writes code for a new feature or bug fix, they create a corresponding unit test to validate the individual function or component before raising a pull request.
- **Unit Testing:** Automated unit tests are executed in the GitHub Actions pipeline when a branch is being merged into develop or main. The tests confirm whether functions work as expected. Any failures prevent the code from being merged.
- **Integration Testing:** Once unit tests pass, integration tests are executed against the test database container. These tests validate that the full request lifecycle functions correctly across all internal layers.

- **Build Check:** A Next.js production build and a FastAPI startup check are run to confirm the application compiles and initialises without errors.
 - **End-to-End Testing:** Cypress E2E tests are executed manually against the staging deployment prior to each demo submission to validate complete user journeys through the browser. E2E tests are not included in the automated CI pipeline due to their dependency on a live staging environment.
 - **Review and Feedback:** If any issues are identified at any stage, feedback is shared among the team. The team members responsible for the failing tests must resolve them before the pull request can proceed.
 - **Deployment:** Once all tests pass and issues are resolved, the application is deployed to the production environment by merging the code into the main branch, which triggers the deployment workflow automatically.
-

4. Quality Assurance Testing

Quality assurance testing is a critical aspect of the PenFlow development lifecycle, focusing on ensuring that the application not only functions as intended but also meets the required standards of performance, scalability, and security. Given that PenFlow handles external vulnerability scanning and sensitive security findings, it is essential to assess how well the software performs under various conditions and to verify that it resists common attack vectors.

4.1 Performance

4.1.1 Objective

The objective of performance testing is to validate that the PenFlow application meets specified response time and throughput requirements under normal and peak load conditions. This includes measuring API response times, scan job completion times, and identifying performance bottlenecks in the scan pipeline.

4.1.2 Performance Metrics

Performance testing focuses on the following key metrics:

- **API Response Time:** Target P95 response time under 2 seconds for all REST API endpoints under normal operating conditions.
- **Scan Completion Time:** A Phase 1 CTEM scan must complete within 60 seconds for 90% of requests, bounded by the slowest single OSINT lookup rather than the sum of all lookups.
- **Throughput:** The system must sustain a minimum of 50 concurrent scan requests without degradation.
- **Error Rate:** Less than 5% error rate under normal load conditions.

4.1.3 Testing Framework

Performance testing is conducted using k6 load scripts for measuring API throughput and response times. WebSocket latency for real-time scan progress updates is measured separately against the staging deployment.

4.1.4 Test Scenarios

Performance tests cover the following critical system operations:

- Phase 1 CTEM scan initiation and parallel OSINT lookup execution
- Phase 2 vulnerability scan job dispatch and worker execution
- Scan report retrieval under concurrent user load
- Domain verification token confirmation endpoint
- Scan history pagination queries

4.1.5 Acceptance Criteria

Performance tests must demonstrate:

- P95 API response time under 2 seconds for all endpoints
- Phase 1 scan completion bounded by the slowest single OSINT source
- No memory leaks or resource exhaustion under sustained load
- WebSocket progress events delivered within 1 second of task state change

4.2 Scalability

4.2.1 Objective

The objective of scalability testing is to evaluate the PenFlow system's ability to handle increasing numbers of concurrent users and scan jobs while maintaining acceptable performance levels. This ensures the platform can grow with client demand without requiring major architectural changes.

4.2.2 Scalability Metrics

Scalability testing measures:

- **Concurrent Users:** System capacity under 100 or more simultaneous authenticated users.
- **Concurrent Scans:** Number of Phase 2 scan jobs that can execute simultaneously in isolated worker containers without cross-client data leakage.
- **Peak Load Handling:** System behaviour during traffic spikes such as multiple clients initiating scans simultaneously.
- **Graceful Degradation:** Performance degradation patterns when OSINT sources are rate-limited or unavailable.

4.2.3 Testing Framework

Scalability testing is conducted using K6 for load and stress testing against the staging API. Test scripts simulate realistic traffic patterns including scan initiation, report retrieval, and scan history queries across multiple concurrent virtual users.

4.2.4 Load Testing Scenarios

Scalability tests simulate the following user behaviour patterns:

- **Gradual Load Increase:** Ramp from 10 to 100 concurrent users over a defined test duration to identify the point of degradation.
- **Sustained Load:** 50 concurrent users maintaining activity for 5 minutes to test system stability.

- **Burst Load:** Sudden spike to 100 or more users to simulate peak demand around a scheduled scan trigger window.
- **Partial Failure Scenario:** One or more OSINT sources return errors during load to verify graceful partial result handling.

4.2.5 Performance Thresholds

Scalability tests must demonstrate:

- System stability with 100 concurrent users
- Response time degradation of less than 50% under peak load compared to baseline
- No cross-client data leakage under concurrent scan execution
- Successful graceful partial results when individual OSINT sources fail during load

4.3 Security

4.3.1 Objective

The objective of security testing is to identify vulnerabilities and security weaknesses in the PenFlow application. Given the sensitive nature of security finding data and the domain ownership verification requirement, it is critical to test for common attack vectors, data protection mechanisms, and access control enforcement.

4.3.2 Security Test Coverage

Security testing covers the following areas:

- **Authentication Bypass:** Testing unauthorized access to authenticated endpoints without a valid JWT token.
- **Authorisation Flaws:** Verifying that a user cannot access another user's scan history, findings, or reports (IDOR prevention) - ownership is enforced via a user_id-scoped lookup, so a cross-user request returns 404 rather than an explicit 403.
- **Domain Verification Bypass:** Verifying that a Phase 2 scan cannot be initiated against an asset that has not been verified.
- **Input Validation:** Testing for SQL injection, XSS, and malformed domain input handling.
- **Rate Limiting:** Verifying that the IP-based rate limiter blocks more than 3 Phase 1 scan submissions per IP per 10 minutes.
- **Secrets Exposure:** Confirming that API keys, database credentials, and environment variables are not exposed in API responses or logs.
- **Audit Trail Integrity:** Verifying that audit log records cannot be modified or deleted by any user role.

4.3.3 Testing Framework

Security testing is conducted using OWASP ZAP for automated vulnerability scanning and k6 scripts for API security checks, targeting the staging deployment. Tests assert correct HTTP status codes, error response shapes, and the absence of sensitive data in responses.

4.3.4 Security Test Scenarios

Security tests evaluate the following attack vectors:

- Requesting an authenticated endpoint with no Authorization header
- Requesting another user's scan status using no/mismatched auth
- Initiating a Phase 2 scan against an unverified asset
- Submitting malformed or oversized domain input to the scan endpoint
- Submitting more than 3 scan requests from the same IP within a 10-minute window
- Attempting to delete or update an audit log record directly via SQL

4.3.5 Security Acceptance Criteria

Security tests must demonstrate:

- All unauthenticated requests to protected endpoints return 401
- All cross-user data access attempts return 404 (ownership-scoped lookup, not an explicit 403 check)
- Phase 2 scan initiation against an unverified asset returns 403
- Rate limiter returns 429 on the fourth scan submission from the same IP within 10 minutes
- No sensitive data (API keys, passwords, internal IDs) present in any API error response
- Audit log records cannot be modified or deleted at the database level

5. End-to-End Testing Policy

5.1 Objective

The objective of end-to-end testing is to validate complete user journeys through the PenFlow application from the browser, ensuring that the frontend, backend, worker pipeline, and database interact correctly as an integrated system. No mocks are used at any layer during E2E testing.

5.2 Framework

E2E tests are executed manually against the staging deployment. Results are documented and stored as test evidence prior to each demo submission.

5.3 Test Scenarios

The following use cases must have passing E2E tests for :

Use Case	Phase
Initiate CTEM scan as guest	1
View inline scan summary	1
log in	1
View scan history (authenticated)	1
Initiate domain verification	2
Initiate Phase 2 vulnerability scan	2
View Phase 2 scan results	2

Use Case	Phase
add/delete domain	2
filter findings domain	2

Each E2E test must include assertions on page navigation, visible UI content such as headings and status badges

5.4 Acceptance Criteria

Definition of Done:

- It has at least one passing integration test covering the happy path.
- It has at least one passing integration test covering a relevant error path such as invalid input or unauthorized access.
- It has a passing E2E test simulating the complete user flow.
- No test in the suite mocks the database or any internal service.
- The CI pipeline is green on the main branch at demo time.

6. Defect Management

6.1 Defect Reporting

When a test fails or a bug is identified, the following process must be followed:

1. A GitHub Issue is opened with the label bug and assigned to the responsible team member.
2. The issue is linked to the failing test or use case.
3. The fix is implemented in a branch named using the convention fix/issue-number-short-description.
4. The fix must include a regression test that would have caught the original defect.
5. The pull request is reviewed by at least one other team member before merging.

6.2 Severity Classification

Severity	Definition	Target Fix Time
Critical	Core use case broken, demo blocked	Same day
High	Feature broken but workaround exists	Within 2 days
Medium	Minor functional issue, feature partially works	Before demo
Low	Cosmetic or non-functional issue	Best effort